EST-744

Clasificación de texto con redes neuronales

Juan Zamora O.

Deep Learning





El Problema con el Promediado

Promediar embeddings ignora la **estructura de secuencias**:

```
"No me gustó esta película"
"Me gusta esta película"

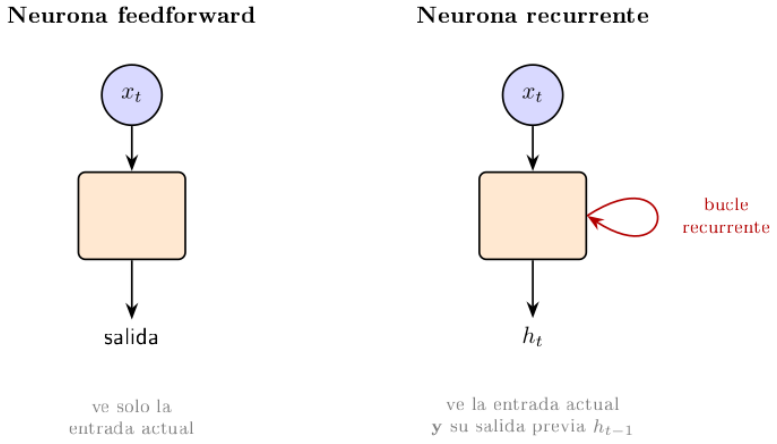
Los embeddings promediados son casi idénticos, ¡pero significan lo opuesto!
```

¿Por qué? El orden de las palabras importa. El contexto importa.

Solución: Usar modelos que procesen secuencias paso a paso

¿Qué es una Neurona Recurrente?

Una neurona ***feedforward*** (Clase 1) solo mira la entrada actual. Una **neurona recurrente** además mira su **propia salida del paso anterior**:



El Mecanismo Paso a Paso

En cada instante de tiempo t, la neurona combina **dos fuentes de información**:

Fuente	Símbolo	Significado
Entrada actual	x_t	La palabra que estamos leyendo ahora
Estado anterior	h_{t-1}	El resumen de todo lo leído antes

Las tres operaciones de la neurona recurrente:

1. **Combinar**: mezcla la entrada nueva x_t con la memoria previa h_{t-1}

$$z_t = W_x \cdot x_t + W_h \cdot h_{t-1} + b$$

2. **Activar**: aplica una no linealidad (típicamente tanh)

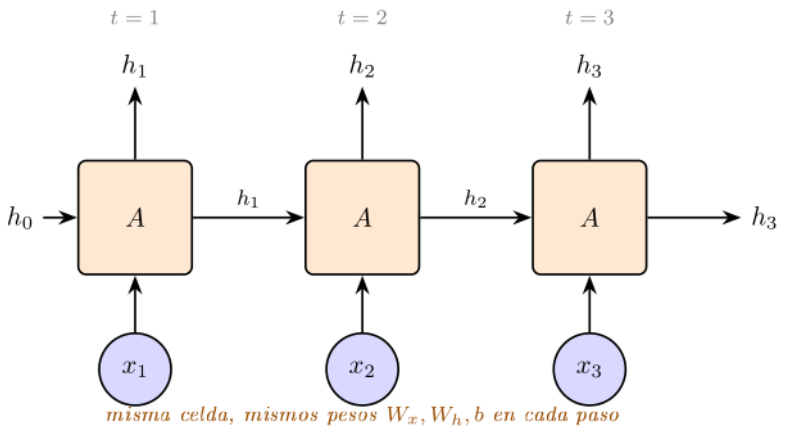
$$h_t = \tanh(z_t)$$

3. **Propagar**: h_t se convierte en la memoria para el paso $t + 1$

El artículo fundacional es Elman (1990), “Finding Structure in Time”, Cognitive Science 14(2). Introduce la RNN “simple” o “de Elman”.

Desplegando la Recurrencia en el Tiempo

El bucle es difícil de visualizar. El truco es **“desplegarlo”** (***unrolling***) y dibujar una copia de la neurona por cada paso temporal.



Analogía: como leer un libro con un solo cerebro. No usas un cerebro distinto por página; el mismo cerebro actualiza su comprensión página a página.

Visualización clásica: Olah (2015), “Understanding LSTM Networks”, blog `colah.github.io`. Para un tratamiento intuitivo, Karpathy (2015), “The Unreasonable Effectiveness of Recurrent Neural Networks”.

La Idea Central

Una RNN mantiene un **estado oculto** que evoluciona mientras lee palabras:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

```
"Me encantan los perros"

h_0 (inicio) → h_1 (tras "Me") → h_2 (tras "encantan") → h_3 (tras "perros")
               ↓                 ↓                      ↓
               salida_1          salida_2               salida_3
```

- **Estado oculto** h_t: resume "lo que sabemos hasta ahora"
- **Nueva palabra** x_t: actualiza el estado oculto
- **Recurrencia**: mismos pesos en cada paso (compartición de parámetros)

RNN vs. Red Feedforward

- **Feedforward** (Clase 1): procesa todas las entradas independientemente
- **RNN**: procesa entradas secuencialmente, el estado lleva información hacia adelante

¿Por qué las RNN tienen dificultades con secuencias largas?

Al retropropagar por muchos pasos temporales, los gradientes se reducen:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_T} \cdot \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

Si cada derivada es < 1, el producto se vuelve diminuto (se desvanece)

Consecuencias: - Los estados iniciales no se actualizan correctamente - No se pueden aprender dependencias a largo plazo - “Fui a la escuela ayer. … La película fue excelente” (olvidó contexto)

Solución: Celdas LSTM (compuertas para controlar el flujo de información)

La Celda LSTM

Las LSTM añaden **compuertas** para controlar qué información mantener/descartar:

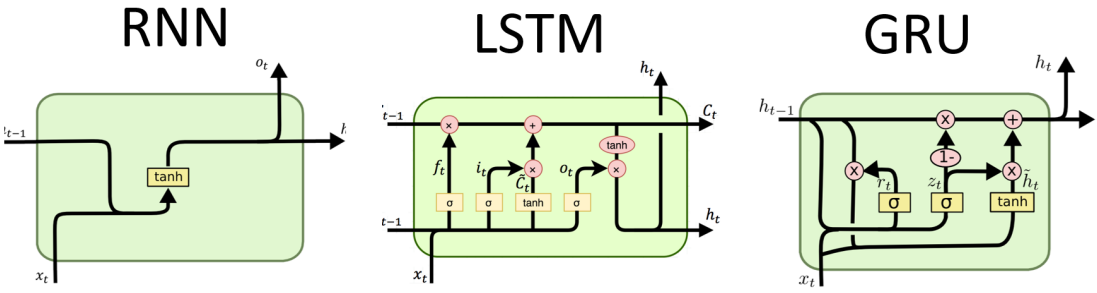
```
[Entrada] → [Compuerta de olvido] → [Compuerta de entrada] → [Estado celular] →
                                  ↓ ↓ ↓                 actualizaciones     ↓
                                 Actualizar           c_t                h_t
```

Tres Compuertas (Intuitivo)

Compuerta	Rol	Pregunta
Olvido	Borra información antigua	“¿Debo olvidar el contexto anterior?”
Entrada	Acepta nueva información	“¿Es importante la nueva palabra?”
Salida	Selecciona la salida	“¿Qué debo sacar ahora?”

Beneficio clave: Los gradientes fluyen mejor a través de compuertas (aditivos, no multiplicativos)

LSTM y GRU



Comparación de Código

```python
# RNN Simple
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.SimpleRNN(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])

# LSTM (mejor)
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.LSTM(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])
```

`return_sequences`: - `True`: retorna estado oculto en cada paso temporal (para apilar) - `False`: retorna solo el estado oculto final (para clasificación)

CNN para NLP: ¿Por qué?

Las CNN se diseñaron para imágenes pero funcionan bien para texto:

Idea: Usar filtros para detectar patrones de n-gramas

```
"La película fue absolutamente brillante"

Filtro (tamaño 2): detecta bigramas
"La película" → combinación positiva
"fue absolutamente" → sentimiento fuerte
```

- El filtro convolucional se desliza sobre secuencias de palabras
- Cada filtro aprende diferentes patrones
- Múltiples filtros capturan diferentes características (n-gramas)

CNN vs. RNN para Texto

Aspecto	RNN/LSTM	CNN
Largo alcance	(ok) Bueno (mediante compuertas)	(!) Depende del tamaño de filtro
Velocidad	(!) Secuencial (lento)	(ok) Paralelo (rápido)
Detección n-grama	(ok) Implícita	(ok) Explícita
Eficiencia param.	(ok) Menos parámetros	(!) Muchos filtros

Tarea de Análisis de Sentimiento (Revisitada)

Objetivo: Clasificar reseñas de IMDb como positivas/negativas

Datos: 25.000 entrenamiento, 25.000 prueba

Tres enfoques para comparar: 1. Embeddings promediados + Dense (baseline de Clase 2) 2. LSTM 3. CNN

Construir un Clasificador LSTM

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Cargar conjunto de datos IMDb
(x_train, y_train), (x_test, y_test) = \
    keras.datasets.imdb.load_data(num_words=5000)

# Índice inverso de palabras para recuperar palabras
word_index = keras.datasets.imdb.get_word_index()
reverse_word_index = dict([(value, key) for (key, value)
                           in word_index.items()])

def decode_review(review_sequence):
    return ' '.join([reverse_word_index.get(i - 3, '?')
                     for i in review_sequence])

# Rellenar secuencias a longitud fija
max_length = 200
x_train_padded = pad_sequences(x_train, maxlen=max_length)
x_test_padded = pad_sequences(x_test, maxlen=max_length)

print(f"Forma rellenada: {x_train_padded.shape}") # (25000, 200)
```

Definir y Entrenar

```
# Construir modelo LSTM
model = keras.Sequential([
    keras.layers.Embedding(input_dim=5000,
                           output_dim=128,
                           input_length=max_length),
    keras.layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Entrenar
history = model.fit(
    x_train_padded, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
    verbose=1
```

CNN para Clasificación de Sentimientos

```
# Modelo CNN con múltiples tamaños de filtro
model = keras.Sequential([
    keras.layers.Embedding(input_dim=5000,
                           output_dim=128,
                           input_length=max_length),

    # Múltiples capas convolucionales con diferentes tamaños de filtro
    keras.layers.Conv1D(filters=128, kernel_size=3,
                        activation='relu'),
    keras.layers.GlobalMaxPooling1D(),

    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

history = model.fit(
    x_train_padded, y_train,
```

Comparar Enfoques

```
# Comparar tres modelos en conjunto de prueba
resultados_modelos = {
    'Embeddings Promediados': 0.88,
    'LSTM': 0.92,
    'CNN': 0.91
}

import matplotlib.pyplot as plt
plt.bar(resultados_modelos.keys(), resultados_modelos.values())
plt.ylabel('Precisión')
plt.title('Resultados de Clasificación de Sentimientos')
plt.ylim([0.85, 0.95])
plt.show()
```

Resultados típicos: - Embeddings promediados: ~88% - LSTM: ~92% - CNN: ~91%

RNN captura orden de palabras; CNN es más rápido pero ligeramente inferior

Relleno y Enmascaramiento

Problema: Las reseñas tienen diferentes longitudes (50-2000 palabras)

Solución 1: Relleno

```
x_padded = pad_sequences(x, maxlen=200, padding='post')
# Reseñas cortas: relleno con ceros al final
# Reseñas largas: truncar a 200 palabras
```

Solución 2: Enmascaramiento

```
keras.layers.Masking(mask_value=0.0)
# Decirle a la red: ignora los ceros rellenos en el cálculo
```

Compensación: - Longitud fija: simple, pero pierde información - Relleno: añade ruido (ceros) - Enmascaramiento: dice al modelo ignorar el relleno

Contexto de Ambas Direcciones

LSTM estándar lee de izquierda a derecha:

```
"La película fue muy buena" → estados ocultos hacia adelante
```

LSTM Bidireccional: Lee en ambas direcciones

```
Adelante:  La → película → fue → muy → buena
                                         ↓
Atrás:     buena ← muy ← fue ← película ← La
```

Concatenar adelante + atrás → contexto más rico

```
keras.layers.Bidirectional(
    keras.layers.LSTM(64)
)
```

Múltiples Capas LSTM

Los modelos más profundos aprenden patrones más abstractos:

```
model = keras.Sequential([
    keras.layers.Embedding(5000, 128),
    keras.layers.Bidirectional(
        keras.layers.LSTM(64, return_sequences=True)
    ),
    keras.layers.Bidirectional(
        keras.layers.LSTM(32, return_sequences=False)
    ),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Nota: `return_sequences=True` → pasar todos los pasos temporales a la siguiente capa

Lo Que Hemos Aprendido

ok Las RNN procesan secuencias paso a paso

ok Las LSTM resuelven el problema de gradientes desvanecientes con compuertas

ok Las CNN detectan patrones de n-gramas eficientemente

ok Relleno y enmascaramiento para longitudes variables

ok Procesamiento bidireccional captura contexto en ambas direcciones

ok Apilar capas aumenta la capacidad

Comparación de Rendimiento

- Embeddings promediados: baseline, rápido, interpretable
- LSTM: mejor precisión, más lento, captura contexto a largo plazo
- CNN: buena precisión, más rápido, enfocado en n-gramas

Interpretar Decisiones LSTM

```python
# Extraer estados ocultos (requiere modificar modelo)
outputs = model.layers[2].output
attention_model = keras.Model(
    inputs=model.input,
    outputs=outputs
)

# Obtener pesos de atención
hidden_states = attention_model.predict(x_test_padded[:10])

# Visualizar qué palabras importan
import matplotlib.pyplot as plt
plt.imshow(hidden_states[0])
plt.title('Estados Ocultos Sobre la Secuencia')
plt.colorbar()
plt.show()
```

Intuición: Se puede ver qué pasos temporales tuvieron cambios grandes en el estado

Solución de Problemas

Problema	Causa	Solución
Entrenamiento lento	Naturaleza secuencial de RNN	Usar CNN o Transformers (próxima clase)
Malos resultados LSTM	Gradientes desvanecientes persisten	Añadir dropout, regularización
Sobreajuste	Demasiados parámetros	Reducir tamaño LSTM, añadir Dropout
Error de memoria	Secuencias largas	Reducir max_length, usar procesamiento por lotes

Tareas Prácticas

1. **Comparación de arquitecturas**: Entrenar los tres modelos (promediado, LSTM, CNN) lado a lado
2. **Bidireccional**: Comparar LSTM unidireccional vs. bidireccional
3. **Apilar capas**: Añadir una segunda capa LSTM. ¿Ayuda? ¿Cuándo perjudica?
4. **Longitud de secuencia**: Variar max_length (100, 200, 500). ¿Compensación precisión vs. velocidad?
5. **Ajuste de dropout**: Probar tasas de dropout 0.0, 0.2, 0.5. ¿Cuál previene sobreajuste mejor?
6. **Afinación de embeddings**: Entrenar embeddings vs. usar congelados pre-entrenados
7. **Predecir e interpretar**: Elegir una reseña, visualizar qué palabras influyeron en la predicción

Preguntas y Discusión

¡Gracias!

Conclusión clave: La estructura de la secuencia importa. Las RNN/LSTM la capturan, pero son lentas. Las CNN son rápidas pero limitadas. La atención resuelve ambos problemas.

Siguiente: El mecanismo de atención que impulsa el NLP moderno

Referencias Completas

Para que los estudiantes profundicen por su cuenta:

- **Goodfellow, I., Bengio, Y. & Courville, A. (2016).** ***Deep Learning***. MIT Press. Capítulo 10: "Sequence Modeling: Recurrent and Recursive Nets". Libro de texto estándar, gratuito en `https://www.deeplearningbook.org`.
- **Elman, J. L. (1990).** "Finding Structure in Time". ***Cognitive Science***, 14(2), 179–211. Artículo fundacional que introduce la RNN simple (o "de Elman").
- **Olah, C. (2015).** "Understanding LSTM Networks". Blog ***colah's blog***: `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`. La explicación visual más citada sobre RNN y LSTM.
- **Karpathy, A. (2015).** "The Unreasonable Effectiveness of Recurrent Neural Networks". `https://karpathy.github.io/2015/05/21/rnn-effectiveness/`. Complemento motivacional con ejemplos sorprendentes de generación de texto.
- **Werbos, P. J. (1990).** "Backpropagation Through Time: What It Does and How to Do It". ***Proceedings of the IEEE***, 78(10), 1550–1560. Para quienes quieran el detalle